

Blockchain Practical - 2

Question 1:

Write a program to take **5 values as input from the user** and store them in a list. Display all the elements of the list and find and display the **hash code** of the list.

Code:

```
import hashlib
user_input = []

for i in range(5):
    item = input(f"Enter data {i + 1}: ")
    user_input.append(item)

print("\nUsing sha512 algo")
for data in user_input:
    hash_value_sha512 =
hashlib.sha512(data.encode()).hexdigest()
    print(hash_value_sha512)
    print("sha512 algo: ", len(hash_value_sha512))
```

Output:

→ hash git:(main) X python3 blockchain.py

Enter data 1: dheeraj

Enter data 2: niharika

Enter data 3: niraj

Enter data 4: saurabh

Enter data 5: najiya

Using sha512 algo

267f3cf29dc92acbf5f96a7dff383a9f7e63062f84185f2022638288072f7ac7bd1c9f1105789

1840dfb2dbf5266f843735a28d73b14357f1cc5e67b17c744c9

sha512 algo: 128

399b17cfb6ae94d17aa64a8a2ccc933161a19bb4ab401c8315bde09ea57a4591214b0ba9b
1011bf8096ce65042a5cb0cdace6f26caa7f5bc1bd0eb7cacef8e28

sha512 algo: 128

06564e38734522879cbb0e54723d4d4408dfbbad4bdcdbde0f9b453a97e79d164d040a428
6922b13ec6e9982c497412832b60c6c65d0761bdee1d0a1a1073e7f

sha512 algo: 128

6df2c5621856fa2cd5e582f33446de3a9c5df340e6cb9a03e517554889bf7956d2d8cc380c7
ef859525be0bd4cbe3c615c3bbcc51bd773a360e768e5ec0b8498

sha512 algo: 128

74a2d3ffba2ee5a2b2fd59e1b1c734edfe456ff70ce05713e57b03ed91a2edc01f4710fb063c
6869410d17cd64cfe487df611170506e274fef095fbd0879b886

sha512 algo: 128

→ hash git:(main) ✕

Question 2:

Write a complete program to demonstrate the basic structure of a **Blockchain** by creating a class named `Block`.

The `Block` class should contain the following data members:

- `data`
- `previousHash`
- `hash`

Also, create a method named `calculateHash()` to calculate the hash value of the block.

Create **3 objects of the `Block` class**, representing **three blocks in a blockchain**. Initialize the required values for each block and establish the connection between consecutive blocks using `previousHash`.

Finally, display for each block:

- Block data
- Previous hash
- Current hash
- Hash length

The output should clearly show the relationship between the **three blocks** and their respective hash values.

Code:

```
import hashlib

class Block:
    def __init__(self, data, previousHash):
        self.data = data
        self.previousHash = previousHash
        self.hash = self.calculate()

    def calculate(self):
        text = self.data + self.previousHash
        self.hash =
        hashlib.sha256(text.encode()).hexdigest()
        return self.hash

block1 = Block("developer", "000")
block2 = Block("engineer", block1.hash)
```

```

block3 = Block("software developer", block2.hash)

print("==== Block 1 ====")
print("Block Data      :", block1.data)
print("Previous Hash:", block1.previousHash)
print("Current Hash   :", block1.hash)
print("Hash Length    :", len(block1.hash))

print("\n==== Block 2 ====")
print("Block Data      :", block2.data)
print("Previous Hash:", block2.previousHash)
print("Current Hash   :", block2.hash)
print("Hash Length    :", len(block2.hash))

print("\n==== Block 3 ====")
print("Block Data      :", block3.data)
print("Previous Hash:", block3.previousHash)
print("Current Hash   :", block3.hash)
print("Hash Length    :", len(block3.hash))

```

Output:

→ hash git:(main) X python3 block.py

==== Block 1 =====

Block Data : developer

Previous Hash: 000

Current Hash : 0a7106d4f725440aea70ab376457f5f76dcb4fd8985cf9c90b73cdcfa9933f1c

Hash Length : 64

==== Block 2 =====

Block Data : engineer

Previous Hash: 0a7106d4f725440aea70ab376457f5f76dcb4fd8985cf9c90b73cdcfa9933f1c

Current Hash : 00e9be09dc81e96ae0e5a780cf35e26cbd282976b035c2f628a8954984d8231f

Hash Length : 64

==== Block 3 =====

Block Data : software developer

Previous Hash: 00e9be09dc81e96ae0e5a780cf35e26cbd282976b035c2f628a8954984d8231f

Current Hash : 097d0e4d96b2e8a83fd8814ceab54f17a56ce8c94ca604019d97df2d85c41a32

Hash Length : 64

→ hash git:(main) X